

Min-Entropy Estimator Comparison

Ahmad Soltani

Supervisor : Didrik Bergström
Examiner : Onur Günlü

Upphovsrätt

Detta dokument hålls tillgängligt på Internet - eller dess framtida ersättare - under 25 år från publiceringsdatum under förutsättning att inga extraordinära omständigheter uppstår.

Tillgång till dokumentet innebär tillstånd för var och en att läsa, ladda ner, skriva ut enstaka kopior för enskilt bruk och att använda det oförändrat för ickekommersiell forskning och för undervisning. Överföring av upphovsrätten vid en senare tidpunkt kan inte upphäva detta tillstånd. All annan användning av dokumentet kräver upphovsmannens medgivande. För att garantera äktheten, säkerheten och tillgängligheten finns lösningar av teknisk och administrativ art.

Upphovsmannens ideella rätt innefattar rätt att bli nämnd som upphovsman i den omfattning som god sed kräver vid användning av dokumentet på ovan beskrivna sätt samt skydd mot att dokumentet ändras eller presenteras i sådan form eller i sådant sammanhang som är kränkande för upphovsmannens litterära eller konstnärliga anseende eller egenart.

För ytterligare information om Linköping University Electronic Press se förlagets hemsida <http://www.ep.liu.se/>.

Copyright

The publishers will keep this document online on the Internet - or its possible replacement - for a period of 25 years starting from the date of publication barring exceptional circumstances.

The online availability of the document implies permanent permission for anyone to read, to download, or to print out single copies for his/hers own use and to use it unchanged for non-commercial research and educational purpose. Subsequent transfers of copyright cannot revoke this permission. All other uses of the document are conditional upon the consent of the copyright owner. The publisher has taken technical and administrative measures to assure authenticity, security and accessibility.

According to intellectual property law the author has the right to be mentioned when his/her work is accessed as described above and to be protected against infringement.

For additional information about the Linköping University Electronic Press and its procedures for publication and for assurance of document integrity, please refer to its www home page: <http://www.ep.liu.se/>.

Abstract

A min-entropy is defined as a conservative measure of uncertainty, which corresponds to the probability of guessing the most common output in a dataset. This thesis evaluates the performance of two min-entropy estimators defined in NIST Special Publication 800-90B, i.e. MultiMCW estimator and the Compression estimator. The comparison focuses on two key metrics: accuracy and execution time for estimating min-entropy values across datasets with known true min-entropy. To ensure a fair comparison, both estimators were tested under identical conditions using synthetic datasets. All tools, configurations, and environments were consistent throughout the experiments. The motivation for this work is the increasing demand for reliable true random number generation (TRNG), especially the Quantum Random Number Generators (QRNGs), where accurate entropy estimation can be helpful for ensuring the randomness quality and security.

Acknowledgments

I would like to give my deepest thankfulness to my supervisor, Didrik Bergström, for his guidance and constructive feedback during the thesis process. I am also truly thankful to my examiner, Onur Günlü, for his valuable comments and suggestions, as well as for providing the opportunity to do this interesting thesis. I would also like to thank the Department of Electrical Engineering (ISY) at Linköping University for providing the opportunity and academic environment that made this work possible. Finally, my thanks go to my opponent for his thoughtful and critical feedback, which helped improve the quality of this work.

Contents

Abstract	iii
Acknowledgments	iv
Contents	v
1 Introduction	1
1.1 Motivation	1
1.2 Aim	2
1.3 Research Questions	2
1.4 Delimitations	2
2 Theory and Related Works	3
2.1 IID and Non-IID	3
2.2 Entropy and Min-Entropy	3
2.3 NIST SP 800-90B	5
2.4 Min-Entropy Estimator	5
3 Method	7
3.1 Implementation Environment and Tools	7
3.2 Data Management	8
3.3 Evaluating the Estimators	10
4 Results	12
4.1 Estimated vs. True Min-Entropy	12
4.2 Accuracy vs. Sequence Length	14
4.3 Runtime Scaling vs. Length	15
5 Discussion	17
5.1 Results	17
5.2 Methods	17
5.3 The Work in a Wider Context	18
6 Conclusion	19
Bibliography	20
List of Figures	22
List of Tables	23
A Algorithms	24



1 Introduction

Since the internet has expanded worldwide, digital communications have increased dramatically [1]. This growth has created a crucial need to ensure the confidentiality and integrity of essential data during information transmission. To protect sensitive information from attackers and vulnerabilities across the Internet, various cryptographic techniques such as Secret Key Cryptography (SKC), Public Key Cryptography (PKC), and hashing methods are employed. These techniques incorporate randomness and complex mathematical operations within cryptographic algorithms to make the keys unpredictable to attackers. Randomness in cryptographic systems relies on Random Number Generation (RNG), which is generally categorized into two main groups: Pseudo Random Number Generation (PRNG) and True Random Number Generation (TRNG). PRNG-based methods offer acceptable levels of security and are widely used in modern encryption and decryption applications. However, these methods depend on computational assumptions that may become insufficient when quantum computers come into play, which means that quantum computers could potentially resolve public keys or decrypt data with relative ease [2].

In contrast to PRNGs, the randomness of TRNGs is based on physical phenomena such as electronic noise, chaotic systems, and other unpredictable natural processes. In general, cryptographic operations based on TRNGs provide higher unpredictability than PRNGs [3]. It is shown in [4] that Quantum Random Number Generation (QRNG) is a subset of TRNGs that produce truly random generated numbers by considering the unique properties of physical particles for example, the polarization of photons. Predicting the direction of photons, when it is emitted by a light diode, is, in principle, impossible, even if all previous outputs are known.

1.1 Motivation

The process of generating cryptographic algorithms using PRNGs is relatively faster than those associated with quantum mechanics. However, a key weakness of PRNGs is that they require an initial value called a seed value. Since the algorithm starts with this seed, it can generate a cyclic sequence that eventually repeats, making it potentially vulnerable to attacks, especially if the seed value is exposed. On the other hand, quantum-based algorithms, particularly those using QRNGs methods, produce truly random numbers. These can be used to

create highly secure encryption systems or even to provide the initial seed for PRNGs based algorithms, by increasing their security [5]

Despite the higher costs and time complexity of QRNGs, studying this field is worthwhile due to the growing demand for robust security. This is especially true when protecting highly sensitive data, such as information used in banking systems or government operations. In such cases, more secure algorithms are essential to ensure data protection [3].

1.2 Aim

The goal of this work is to evaluate and compare the performance of two min-entropy estimators: the Multi Most Common Window (MultiMCW) estimator and the Compression estimator [6]. The comparison will focus on two main aspects. The execution time (testing data with different lengths) and the accuracy of the estimators. The specific objectives are as follows.

1. Implement and validate two min-entropy estimators from NIST SP 800-90B: the MultiMCW estimator and the Compression estimator and
2. Implement a synthetic data generator to generate binary sequences with known min-entropy to estimate the accuracy of the two estimators.

1.3 Research Questions

1. What is the accuracy of each estimator for different data lengths?
2. What is the execution time of each estimator as data length increases?

1.4 Delimitations

The main focus of this study is to implement and validate the MultiMCW and Compression estimators, rather than understanding the analysis of the underlying algorithms in a deep manner. It does not focus on providing detailed explanations of the complex mathematical and statistical definitions behind the estimators. Although, the concepts of Independent and Identically Distributed (IID) and Non-IID data are closely related to this work, a detailed explanation of them is not in the scope of this thesis.

To validate the results, a synthetic generator will be implemented, by producing binary sequences with known min-entropy values. A potential weakness of this method is that it may not fully capture the complexity and unpredictability of real inputs, such as those generated by QRNGs.



2 Theory and Related Works

In a recent study by [4], the researchers explored the potential of QRNG, by studying the phenomena associated with perovskite light-emitting diodes (PeLEDs). The random numbers generated by PeLEDs passed the NIST statistical tests [4], these bits have a high min-entropy (more about min-entropy in Section 2.2.2) and provide a strong security against attackers and vulnerabilities. In practical applications, QRNG-generated bits are suitable for use in encryptions and gambling systems [4].

2.1 IID and Non-IID

As described in [7], there are two assumptions about the type of entropy sources: they can either be IID or non-IID. To define an entropy source as IID, two conditions must be met. First, the probability distribution of an observation is the same for each one. And second, the occurrence of one observation does not affect the others. On the other side, if an entropy source does not meet at least either of these conditions, then the source is classified as non-IID. This property is tested by several statistical tests, which are part of the NIST SP 800-90B documentation.

2.2 Entropy and Min-Entropy

The concepts of entropy and min-entropy are crucial in information theory for testing the uncertainty related to the possible outcomes of a random variable. Both metrics measure uncertainty, but they do so using different approaches [7, 8]. These concepts are especially relevant when analysing data generated by a random number generator (RNG). The notations below are formulated for RNG-generated sequences and are used in the discussions of entropy (Section 2.2.1) and min-entropy (Section 2.2.2).

Notation! Let $\mathcal{X} = \{0, 1\}^n$ denote the set of all binary sequences of length n with possible outcomes $\{x_1, x_2, \dots, x_{2^n}\}$. Let S be a discrete random variable taking values set $\mathcal{X}$. Let P be a probability mass function (pmf) on the discrete random variable s .

2.2.1 Entropy

In information theory, entropy is used to calculate the average information content of a random variable. This means that entropy tells us how unpredictable a probability distribution is on average.

Definition 1. The Entropy of P_s is defined as:

$$H(P_s) \triangleq - \sum_{s \in \mathcal{X}} P_s(s) \log_2 P_s(s) \quad (2.1)$$

where we have:

- $P_s(s) = \Pr[S = s]$ and $s \in \mathcal{X}$; and
- $\log_2$ denotes the base-2 logarithm used to measure the data set in bits.

Example 1: Let $\mathcal{X} = \{00, 01, 10, 11\}$ and let P_s be the pmf of a random variable S that has the following distribution:

$$P_s(00) = 0.6, \quad P_s(01) = 0.20, \quad P_s(10) = 0.15, \quad P_s(11) = 0.05.$$

Its Entropy is:

$$\begin{aligned} H(P_s) &= - \sum_{s \in \mathcal{S}} P_s(s) \log_2 P_s(s) \\ &= -[0.60 \log_2 0.6 + 0.20 \log_2 0.20 + 0.15 \log_2 0.15 + 0.05 \log_2 0.05] \\ &\approx -[-0.6(0.7) + 0.20(-2.3) + 0.15(-2.7) + 0.05(-4.3)] \\ &\approx 0.42 + 0.46 + 0.40 + 0.22 \approx 1.5 \text{ bits.} \end{aligned}$$

Definition 2. If P_s is the uniform distribution over $\mathcal{S}$, then each sequence $s \in \mathcal{X}$ has probability:

$$P(s) = \frac{1}{2^n}.$$

In this case, the Entropy is

$$H(P_s) = - \sum_{s \in \mathcal{X}} \frac{1}{2^n} \log_2 \frac{1}{2^n} = n. \quad (2.2)$$

Example 2: Let the distribution in Example 1 be uniform:

$$P(s) = \frac{1}{4} \quad \text{for all } s \in \mathcal{S}.$$

In this case, the Entropy is:

$$H(P) = - \sum_{s \in \mathcal{X}} \frac{1}{4} \log_2 \frac{1}{4} = 4 \cdot \frac{1}{4} \cdot 2 = 2 \text{ bits.}$$

2.2.2 Min-Entropy

According to the guideline documentation [7] published by the National Institute of Standard and Technology (NIST), min-entropy is a conservative measure of the guessability of an entropy source. In contrast to entropy, which measures the average unpredictability of all possible outcomes, min-entropy measures the unpredictability of the most likely outcome.

Definition 3. The min-entropy of the distribution P_s , denoted $H_{\min}(P_s)$, is defined as:

$$H_{\min}(P_s) \triangleq - \log_2 \left(\max_{s \in \mathcal{X}} P_s(s) \right). \quad (2.3)$$

Let P_s be the pmf over all possible binary sequences generated by QRNG. In practice this distribution is unknown. In the best-case scenario, the distribution is perfectly uniform, meaning that each possible outcome has an equal probability. To assess the unpredictability of the distribution, the min-entropy rate denoted as $H_{\min}(S)$ must to be estimated. This is done using min-entropy estimators as classified in NIST SP 800 90B (see Section 2.3 and 2.4).

Example 3: Consider the distribution in Example 1. The most likely sequence is 00, with $P_s(00) = 0.6$, so the min-entropy for this distribution is:

$$H_{\min}(P) = -\log_2(0.6) = 0.737 \text{ bits.}$$

Definition 4. If P_s is the uniform distribution over $\mathcal{S}$, then each sequence $s \in \mathcal{X}$ has probability:

$$P_s(s) = \frac{1}{2^n}.$$

In this case, The Min-entropy is

$$H_{\min}(P_s) = -\log_2\left(\frac{1}{2^n}\right) = n. \quad (2.4)$$

Example 4: Consider the distribution P in Example 1 the Min-Entropy for this distribution is:

$$H_{\min}(P) = -\log_2\left(\frac{1}{4}\right) = 2 \text{ bits.}$$

2.3 NIST SP 800-90B

The National Institute of Standards and Technology (NIST) Special Publication (SP) 800-90B provides guidelines to standardize the quality of entropy sources, making them suitable for cryptographic applications. In 2018, NIST published the new test suite NIST SP 800-90B [7], which consists of a collection of statistical tests and cryptographic algorithms (i.e., estimators). The purpose of this suite is to evaluate various RNG-generated sources to determine whether the source is IID or non-IID. If the source is non IID, the min-entropy estimators are applied to estimate the min-entropy of the corresponding RNG-generated sequences s .

2.4 Min-Entropy Estimator

As described in [9], the NIST SP 800-90B suite runs ten non-IID min-entropy estimators (plus one for IID sources) on the same raw data. It selects the smallest output as the official min-entropy bound. In practice, each min-entropy estimator can disagree with the true min-entropy in different ways. Some estimators, such as the Collision estimator, consistently provide the lowest estimates, while others, like the Most Common Value (MCV), tend to overestimate. According to this study, underestimation of min-entropy is preferred, and any overestimate (i.e., a value above the true min-entropy) is simply discarded.

2.4.1 Multi Most Common Window (MultiMCW) Estimator

The MultiMCW predictor is a collection of sub-predictors used to estimate min-entropy in a non-IID sequence. These sub-predictors are called windows ($w1$, $w2$, $w3$, and $w4$), aiming to predict the most commonly occurring digit based on previous occurrences. The purpose of having multiple windows is to identify repeating patterns within the sequence, which can be useful for cryptographic key combinations.

The algorithm starts by checking the most common value at the index $w1 + 1$ (from the smallest window to the largest window). Each window tracks its own score by counting the

number of correct predictions it makes. After every timescale, the windows shift within the sequence. The window with the highest number of correct predictions becomes the predictor window until another window hits its score and takes the lead to predict. In the case of a tie situation, the window whose most common value has appeared recently is selected as the predictor. *See Appendix A for the pseudocode.*

2.4.2 Compression Estimator

The compression estimator assesses the uncertainty within a sequence by evaluating how compressible the data is, which indicates the presence of patterns and repetitions. A truly random data sequence cannot be compressed, whereas sequences with patterns and redundancy can be compressed significantly. The aim of the estimator is to identify these patterns and reduce the data size by splitting the given data into two disjoint groups. The first group is used by the compression algorithms to create a dictionary containing unique blocks of a certain length. Once the dictionary is complete, these unique blocks are used to identify identical blocks in the second group. The more blocks are detected, the shorter the entire data sequence becomes, indicating a compressible sequence or high predictability [10]. *See Appendix A for the pseudocode.*

2.4.3 Mean Absolute Error (MAE)

To quantify the accuracy of the MultiMCW estimator and Compression estimator as a function of sample size, the Mean Absolute Error (MAE) must be calculated. The MAE for each sequence length L is defined as [11]

$$\text{MAE}(L) = \frac{1}{N} \sum_{i=1}^N |\hat{H}_i(L) - H_{\min}^{\text{true}}| \quad (2.5)$$

where

- $\hat{H}_i(L)$ is the i -th estimator output on an independent sequence of length L ;
- $H_{\min}^{\text{true}}$ is the (known) true min-entropy; and
- N is the number of independent replicates of length L .

According to [11] MAE is a significantly better evaluation approach than the other methods such as Root Mean Square Error (RMSE). The purpose of these statistical approaches is to outline the performance of estimation methods. In this study, the calculation of MAE provides an average of the absolute deviations between the min-entropy estimators and the true min-entropy.



3 Method

This section describes the methodology applied in the implementation and evaluation of two min-entropy estimators (MultiMCW estimator and Compression estimator), based on the guidelines from NIST SP 800-90B. The purpose of this section is to provide a broader overview of both practical and theoretical decisions that shaped the entire process. It covers not only the technical setup and tools applied, but also the deeper reasoning behind each methodological choice. It is important to evaluate how closely the performance of the implemented min-entropy estimator aligns with what the theory suggests. In other words, it is not just about making the code run, but also about ensuring that the estimators behave as they are supposed to, especially under varying data conditions.

Moreover, this section describes the programming side of the project by illustrating and explaining key parts from the implementation. This also includes the choice of programming language, the strategies used to handle data of varying lengths, and the structure of the implementation. Finally, these methodologies were selected to ensure that the project met its primary objectives and answered the research questions.

3.1 Implementation Environment and Tools

The entire programming part of this work is written in C++ programming language. C++ was selected primarily due to its efficiency in low-level operations compared to other options such as Python and Java. This capability offers several advantages, including more direct memory management and more efficient debugging options. Because this work requires effective use of resources like memory and CPU, another reason for selecting C++ is its more straightforward experience when, for example, identifying the most time-consuming parts of the code. vVisual Studio Code (VS Code) is used as text editor because of its user-friendly interface and helpful features like different extensions. The implementation files are shared via the Gitlab platform with both the supervisor and the examiner to facilitate easy feedback and collaboration on the code.

3.1.1 Code Structure

All code-related files are stored in the src directory while data-related files are placed in a separate directory called data-sets. The codebase is organized into separate C++ files for each

min-entropy estimator, MultiMCW-estimator.cpp, Comp-estimator.cpp and Synthetic.cpp. Each file has its own main() function along with the relevant supporting functions. This structure makes it easier to test, debug, and benchmark the performance for each estimator independently. For better efficiency and maintainability, the codebase files could have been separated into header files, but due to time constraints, it is not feasible for this work.

3.1.2 Measuring Execution Time

Measuring the execution time is a crucial aspect of this work. The measurement must be as accurate as possible because the recorded execution times are used to validate the performance of each estimator. To achieve this goal, C++ Standard Template Library (STL) provides the chrono library, which is used in this work. According to the official C++ documentation [12], the chrono library is based on three main functionalities: clocks, time points and durations. In this study, the high-resolution-clock is used because it gives the smallest time resolution (tick period), meaning it offers the highest possible precision available in the corresponding operating system (OS). The actual precision depends on the OS and the underlying hardware running the clock. For instance, Linux-based systems can offer nanosecond precision, while in Windows-based systems provide microsecond to nanosecond precision, depending on the system's scheduling mechanisms. In this work, Windows OS is used. The time measurement is implemented by setting a starting point before the execution of the program within the main function. After running the program associated with the min-entropy estimation is completely run, an end point is recorded. The duration between these two points represents the execution time of the min-entropy estimators for a given data length.

Listing 3.1: Measuring execution time using chrono.

```
auto start = std::chrono::high_resolution_clock::now();
// ... code ...
auto end = std::chrono::high_resolution_clock::now();
double duration = std::chrono::duration<double>(end - start).count();
```

3.1.3 Plotting Tools

To generate the charts presented in Chapter 4, the raw measurement results are structured and saved as CSV files. These CSV files serve as input to Python scripts that produce the plots. Each script uses the pandas library (imported as pd) to read the raw data from CSV files and the matplotlib.pyplot module (imported as plt) to render the charts.

3.2 Data Management

Efficient and consistent data handling is a crucial part of this study. Two different data sources, located in the data-sets directory and in two different formats, are used to run the estimators: (1) the raw binary output data generated by a QRNG, and (2) the synthetic binary sequences in string format that have known min-entropy estimates. Each estimator implementation includes two separate functions for file reading, one for reading the file in binary mode (.bin) and another for string-based text files (.txt). This approach ensures the flexibility of reading files, especially when working with data from different sources. It enables the estimators to be tested effectively under different input formats. The following listings below show the relevant code snippets in the implementation.

Listing 3.2: Reading file in raw binary mode.

```
ifstream file(filename, std::ios::binary);
if (!file.is_open()) {
```

```

    std::cerr << "Error:_Could_not_open_file_" << filename << "\n";
    return {};
}
unsigned char buffer[MAX_BYTES];
file.read(reinterpret_cast<char*>(buffer), MAX_BYTES);

```

Listing 3.3: Reading file in string mode line-by-line.

```

ifstream file(filename);
vector<int> bits;
string line;
while (getline(file, line)) {
    for (char c : line) {
        if (c == '0' || c == '1')
            bits.push_back(c - '0');
    }
}

```

3.2.1 Data Generated by QRNGs

One terabyte (1 TB) of raw binary data, generated by a QRNG, is available for further testing. Running the MultiMCW estimator and compression estimator on such a large dataset can be very time- and resource-intensive. Therefore, the data sets are provided in smaller files. From an implementation perspective, data files are written in smaller blocks to facilitate the reading and eventually the debugging process. Having efficient debugging strategies makes it easier to identify and resolve errors in the code.

3.2.2 Synthetic Data Generation

Although NIST SP 800-90B recommends a sample size of at least 1,000,000 for entropy estimation, it also allows the use of smaller data sets, with a minimum of 1,000 samples. To evaluate the performance of the MultiMCW estimator and the Compression estimator in terms of both speed and accuracy, a synthetic data generator was implemented in C++. This generator was designed to produce binary sequences of controlled lengths and known min-entropy values, making it ideal for validation purposes.

Six data sets with sizes of 128B, 20KB, 45KB, 75KB, 100KB, and 128KB were generated. Each data set was associated with a predefined min-entropy value: 0.15, 0.30, 0.45, 0.60, 0.75, and 0.90, respectively. *See Appendix A for the pseudocode.*

This implementation offers flexibility in generating sequences with varying lengths and entropy levels, making it well-suited for testing the estimators under controlled conditions.

Table 3.1 below presents the results. The first column lists the dataset sizes, the second column shows the corresponding true min-entropy values, the third column provides the estimated values from the MultiMCW estimator, and the fourth column displays the estimates from the Compression estimator.

Data Length (KB)	TrueMinEntropy	MultiMCW	COMP
0.128	0.15	0.08	0.06
20	0.30	0.29	0.17
45	0.45	0.44	0.27
75	0.60	0.59	0.39
100	0.75	0.74	0.54
128	0.90	0.89	0.76

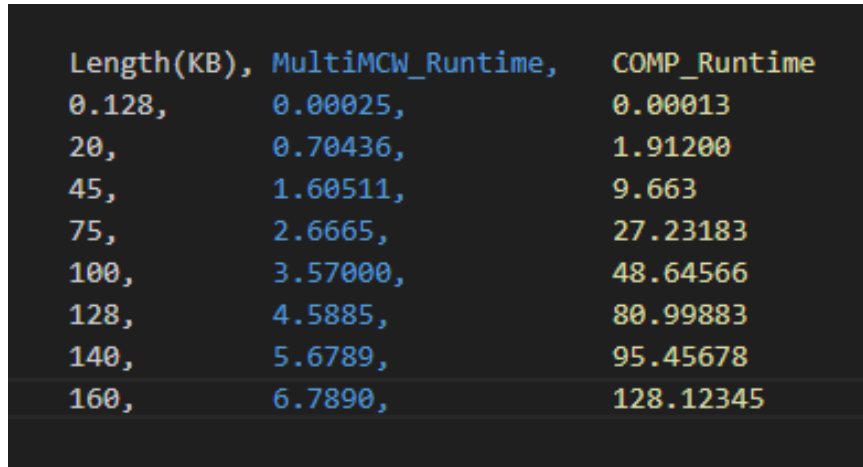
Table 3.1: Estimated and True Min-Entropy vs. Data Lengths

3.3 Evaluating the Estimators

To assess the practical performance of the two min-entropy estimators, their outputs are compared against known true min-entropy values across several datasets of varying lengths. The comparison aims to evaluate how accurately each estimator approximates the true min-entropy under controlled conditions. The results are visualized using charts and tables, providing a clear and intuitive understanding of the relationship between the estimated and actual min-entropy values. These visualizations help highlight each estimator's convergence behavior and accuracy across different data sizes.

3.3.1 Visualization of Execution Time

To analyse and visualize how execution time increases with different data lengths, Figure 4.4 and Table 4.2 are created. For each estimator, synthetic datasets of sizes 128B, 20KB, 45KB, 75KB, 100KB, 128KB, 140KB and 160KB are executed individually. To ensure high precision runtime measurement, a `chrono` timer is initialized at the start of the program. The timer records the start time before execution and end time after running each dataset. Execution time is measured in minutes.



Length(KB),	MultiMCW_Runtime,	COMP_Runtime
0.128,	0.00025,	0.00013
20,	0.70436,	1.91200
45,	1.60511,	9.663
75,	2.6665,	27.23183
100,	3.57000,	48.64566
128,	4.5885,	80.99883
140,	5.6789,	95.45678
160,	6.7890,	128.12345

Figure 3.1: Runtime vs. Data Lengths

Figure 3.1 shows that the execution times for both estimators are low with short input data. The Compression estimator (rightmost column) shows particularly low execution time at 128B, even lower than MultiMCW estimator. But its execution time increases significantly as the data size grows. The MultiMCW estimator (middle column) shows consistently lower execution time compared to the Compression estimator, a point discussed in detail in Section 4.3.

3.3.2 Visualization Accuracy vs. Data Size

To analyse the relationship between MAE and sequence lengths, five datasets are generated with lengths of 10KB, 20KB, 35KB, 50KB and 60KB, all sharing the same true min-entropy value (0.5). By calculating the MAE for each dataset, it becomes possible to observe how closely an estimator's estimation converges to the true min-entropy value. Since MAE is calculated as the average of the absolute differences between the estimated and true min-entropy, it provides a clear measurement for evaluating estimation accuracy. This approach makes it easier to capture how much data is required for each estimator to achieve a certain accuracy.

Length(Bits),	MAE_MCW,	MAE_COMP
81920,	0.011,	0.195
-> (N = 10 at L = 81920)		
163840,	0.007,	0.163
-> (N = 10 at L = 163840)		
286720,	0.006,	0.192
-> (N = 10 at L = 286720)		
409600,	0.005,	0.191
-> (N = 10 at L = 409600)		
491520,	0.004,	0.191
-> (N = 10 at L = 491520)		

Figure 3.2: MAE values vs. Data Lengths

Figure 3.2 illustrates the structure of the dataset used to plot the relationship between estimation accuracy and data length. The data is organized in a CSV file format. Each data set of length L is run N times to calculate the MAE values (Section 2.4.3). The MultiMCW estimator (middle column) consistently maintains a very low MAE across the datasets, while the Compression estimator (rightmost column) exhibits higher MAE values, which is discussed in detail in Section 4.2.



4 Results

This chapter presents the results and findings, illustrated using charts and tables. Section 4.1 gives a comparative explanation of the estimated vs. true min-entropy values. Section 4.2 evaluates the accuracy of MultiMCW estimator and Compression estimator. Finally, Section 4.3 analyzes the execution time and time complexity associated with each estimator.

4.1 Estimated vs. True Min-Entropy

The estimated values provided by the MultiMCW estimator and Compression estimator are compared to the true min-entropy values, shown in Table 3.1. It is evident that the estimated min-entropy values lie below the true min-entropy value, which is also called underestimation. The MultiMCW estimator closely aligns with the true min-entropy across the range of data lengths tested. In contrast, the Compression estimator underestimates min-entropy by a large margin, meaning its estimations deviate significantly from the true min-entropy values, across the data lengths tested. This behaviour demonstrates that the MultiMCW estimator is more reliable when it comes to estimating min-entropy values for Non-IID sources.

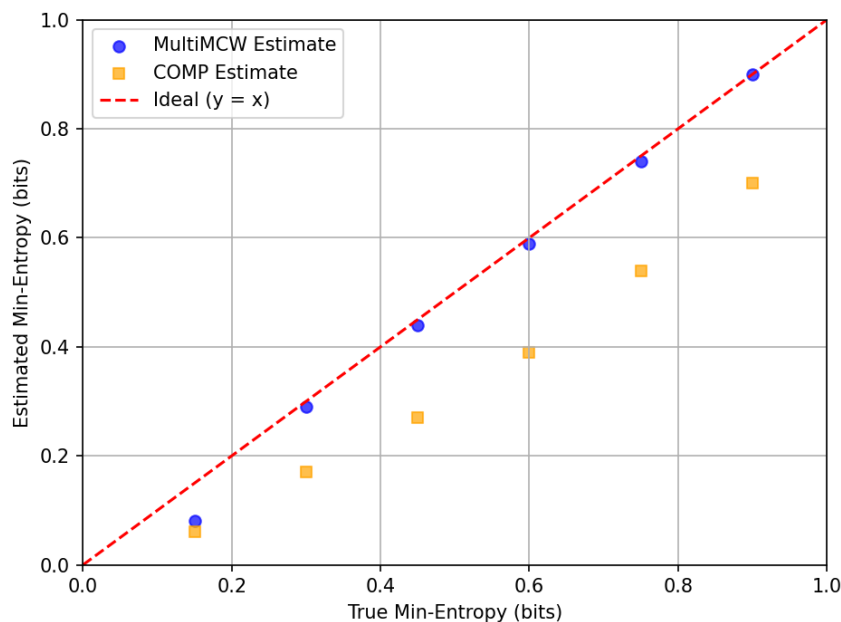


Figure 4.1: Estimated Min-Entropy vs. True Min-Entropy

Figure 4.1 depicts the estimated min-entropy values (in bits) on the Y-axis and the true min-entropy on the X-axis. From the figure, we observe that both estimators tend to underestimate the min-entropy. However, MultiMCW estimator (blue) quickly aligns with the true min-entropy, while the Compression estimator (orange) consistently continues its underestimation.

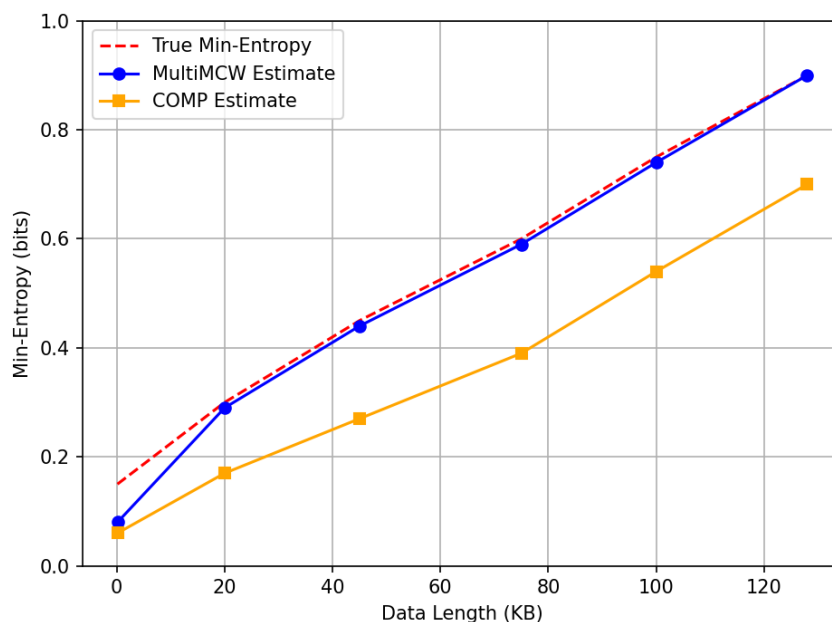


Figure 4.2: Estimated Min-Entropy vs. Data Length

Figure 4.2 depicts the min-entropy values (in bits) on the Y-axis and a range of data lengths (from 128 B to 128 KB) on the X-axis. From the blue line (the MultiMCW estimator) closely

aligns with the red line (representing ideal min-entropy), while the orange line (the Compression estimator) deviates significantly from the ideal case across all data lengths considered.

4.2 Accuracy vs. Sequence Length

To evaluate the accuracy of each estimator, the MAE as calculated over multiple data lengths (see Figure 3.2). The results plotted in Figure 4.3 demonstrate a consistently significant advantage of the MultiMCW estimator compared to the Compression estimator. The MultiMCW estimator maintains an extremely low MAE, remaining approximately 0.005 across all tested data, indicating an exceptional performance regardless of data length, particularly within the first 20 KB data. In contrast, the Compression estimator has relatively high MAE values, varying between approximately 0.15 and 0.20 (see Table 4.1) with a notable dip in error around the 20 KB data length. This variation suggests that the accuracy of the Compression estimator is more sensitive to data lengths.

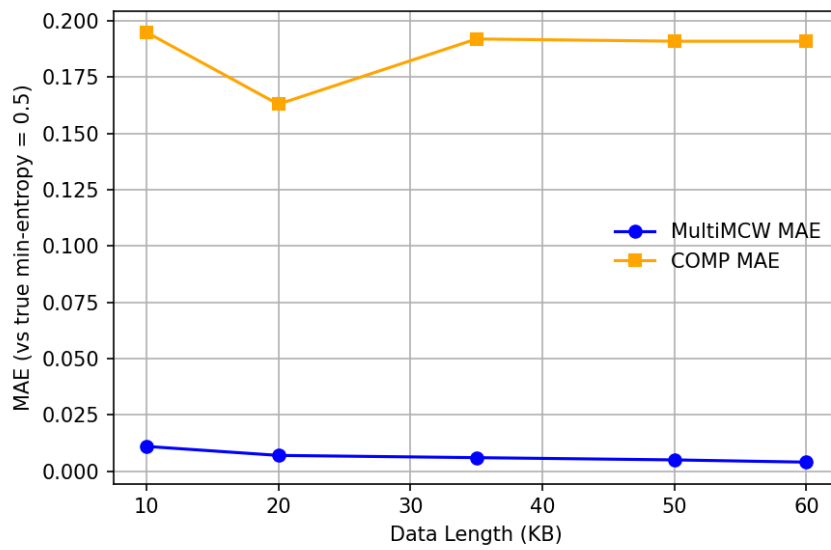


Figure 4.3: MAE comparison MultiMCW Estimator vs. Compression Estimator

In Figure 4.3, the Y-axis represents the MAE values calculated for the true min-entropy 0.5, and X-axis represents the data length. The figure shows that MultiMCW estimator maintains a consistent trend across different data lengths, but the Compression estimator exhibits a dip at data point 20KB. One possible explanation is that, at that point, the estimated min-entropy happens to align more closely with the true min-entropy or alternatively, the dip is the result of a single error in the calculation of MAE.

Estimators with a high accuracy (low MAE) effectively capture the true randomness of the data. In our scenario, the Compression estimator has a lower efficiency in estimating the randomness of a data set. From Table 4.1, it is evident that the variance of the Compression estimator (≈ 0.04) is significantly larger than the MultiMCW estimator (≈ 0.008).

Data Length (KB)	MultiMCW (MAE)	COMP (MAE)
10	0.010	0.200
20	0.005	0.160
30	0.003	0.180
40	0.002	0.190
50	0.002	0.180
60	0.002	0.180
Variance (approximately)	0.008	0.040

Table 4.1: MAE comparison between MultiMCW and Compression estimators.

4.3 Runtime Scaling vs. Length

Execution time, also called runtime, is another crucial aspect for quantifying efficiency of each estimator in practice. It indicates how much time an estimator takes to execute. The lower the runtime, the more efficient the estimator is. Figure 4.4 depicts the runtime scaling of the MultiMCW estimator and the Compression estimator over the data lengths tested in this study. The MultiMCW estimator provides impressive computational efficiency, showing a linear increase in runtime with data length, remaining around 5 minutes even at the largest data size tested (128KB). In contrast, the Compression estimator exhibits an extremely high runtime, increasing exponentially with data length.

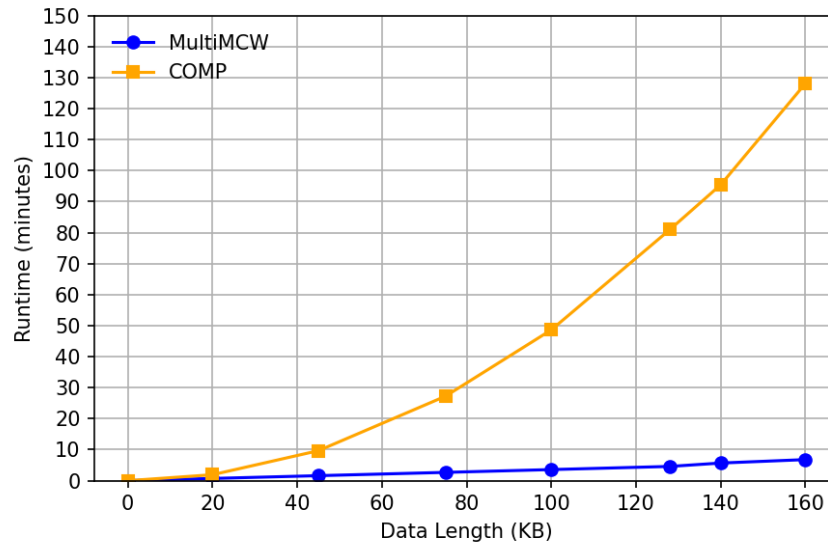


Figure 4.4: Runtime comparison MultiMCW estimator vs. Compression estimator

As illustrated in Figure 4.4 the Y-axis represents the runtime and X-axis represents the data lengths. The figure clearly shows a significant difference in runtime between the two estimators.

4.3.1 Time Complexity over Data Length

Analysing the time complexity provides a crucial understanding of the estimator's scalability in practical usage. Table 4.2 demonstrates the differences in runtime complexity between the MultiMCW and Compression estimators.

Data Length (KB)	MultiMCW (min)	Increase (min)	COMP (min)	Increase (min)
0	0	–	0	–
20	1	+1	2	+2
40	2	+1	9	+7
60	3	+1	19	+10
80	4	+1	32	+13
100	5	+1	50	+18
120	6	+1	72	+22
140	6	+1	96	+24
160	7	+1	128	+32
Time complexity	Linear $O(n)$		Quadratic $O(n^2)$	
Function	$T(n) \approx n/20$		$T(n) \approx n^2/200$	

Table 4.2: Time complexity comparison MultiMCW estimator vs. Compression estimator

From Table 4.2, across data lengths n from 0 KB to 160 KB, the MultiMCW estimator exhibits linear time complexity, $O(n)$, with $T(n) \approx n/20$, while the Compression estimator exhibits quadratic time complexity, $O(n^2)$, with $T(n) \approx n^2/200$, where n denotes the data length in kilobytes.



5 Discussion

5.1 Results

Regarding the calculation of MAE in Section 2.3.4, increasing the number of samples generally leads to higher accuracy and more reliable results. It is noteworthy that all input values including the estimated min-entropy in Figure 4.2, MAE in Figure 4.3, and runtime values in Figure 4.4 were recorded with a precision of three to five decimal points.

The MultiMCW estimator, by contrast, benefits from its window-based logic. As described in NIST SP 800-90B [7], the input data is divided into four fixed windows. This property enhances both the accuracy of prediction and the estimator’s time efficiency.

In a security context, where evaluating the level of predictability and uncertainty within a sequence is critical, the MultiMCW estimator offers a significant advantage, not only due to its high accuracy but also because of its linear time complexity. The Compression estimator, in contrast, might still be a reliable option when data length is not a critical factor. It is important to note that a deep statistical and theoretical understanding of the concepts behind the algorithms were not in the scope of this work.

5.2 Methods

First and foremost, the environment and tools used to implement and validate the two min-entropy estimators were kept consistent to ensure a fair comparison. Both estimators were tested under identical conditions: using the same input data, running on the same hardware, and being implemented in the same programming language. The consistency was essential to ensure the reliability of the results. Any deviation from these conditions could have compromised the validity of the findings. Secondly, the behaviour of the estimators might have varied if larger datasets such as those generated by QRNGs had been tested. However, due to time constraints, testing such large datasets was not feasible within the scope of this study. Although other methodological approaches could have been employed to test the validity of the findings, the methods selected in this study were chosen with confidence, as they delivered results that were reasonable and sufficient. This work has the potential to serve as a foundation for future research at a more advanced level, where more efficient implementations and broader testing across varying data lengths could be prioritized. One promising direction for improvement is applying parallel computing, particularly in optimizing the Compression

estimator, which relies on a binary search function that is computationally expensive. This could lead to faster and more effective performance. The current implementation primarily uses `std::vector`, which is not the most time-efficient option. Replacing these with more optimized structures, such as binary trees or hash tables, could reduce the time complexity and improve overall performance. Additionally, replacing nested loops with more efficient algorithms could further reduce the execution time. Because synthetic data generation plays a central role in this study, exploring more reliable and efficient generation approaches could also be beneficial. Finally, the theoretical framework is primarily based on the NIST SP 800 90B series documentation, in addition to peer-reviewed related works in the field of randomness estimation and cryptographic security.

5.3 The Work in a Wider Context

Robust estimation of min-entropy is a central focus in modern cryptography, as it enables the generation of truly random keys, an essential component in securing real-world applications such as banking networks, election systems and other domains where sensitive data must be protected from attackers. This study considered and expressed a preference for highly accurate and time efficient min-entropy estimator: the MultiMCW estimator, which can be used for practical operations that require unpredictability.



6 Conclusion

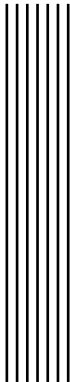
This thesis evaluated two min-entropy estimators: MultiMCW and Compression—by implementing them in the C++ programming language. These estimators are described in NIST SP 800-90B [7]. The evaluation focused on two key aspects: estimation accuracy and runtime scaling, each tested across different data lengths for their respective purposes.

To assess estimation accuracy, we calculated MAE values, using sequences of lengths 10KB, 20KB, 35KB, 50KB, and 60KB, each with a known min-entropy of 0.5, as shown in Figure 4.3 and Table 4.1. To analyse the runtime and time complexity, we recorded execution times for input lengths of 128B, 20KB, 45KB, 75KB, 100KB, and 128KB, presented in Figure 4.4 and Table 4.2.

The key findings are as follows:

- High accuracy and linear runtime for the MultiMCW estimator: The MultiMCW estimator maintains a consistently low MAE variance of approximately 0.008. The runtime measurements across increasing input sizes demonstrate linear time complexity $O(n)$, where the execution time grows proportionally with data length.
- Low accuracy and exponential runtime for the Compression estimator: The Compression estimator exhibits a relatively high MAE variance of around 0.04, and its runtime increases quadratically $O(n^2)$ as the input size increases.

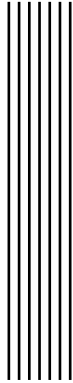
These findings establish the MultiMCW estimator as a more efficient choice when both estimation accuracy and time complexity are critical. In contrast, as illustrated in Figure 3.1, the Compression estimator, despite its exponential time complexity, performs faster on shorter input lengths. It also tends to conservatively underestimate the min-entropy, while the MultiMCW estimator converges closely to the true min-entropy, as shown in Figure 4.2.



Bibliography

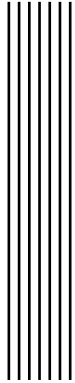
- [1] Dilip Kumar Sharma, Ningthoujam Chidananda Singh, Daneshwari A Noola, Amala Nirmal Doss, and Janaki Sivakumar. "A review on various cryptographic techniques algorithms". In: *ScienceDirect* 51.1 (2022). DOI: <https://doi.org/10.1016/j.matpr.2021.04.583>.
- [2] Vasileios Mavroeidis, Kamer Vishi, Mateusz D. Zych, and Audun Jøsang. "The Impact of Quantum Computing on Present Cryptography". In: *International Journal of Advanced Computer Science and Applications* 9 (2018). URL: <https://arxiv.org/pdf/1804.00200>.
- [3] Prateek Agnihotri and Preeti Agarwal Mittal. "An Efficient approach of True Random Number Generation using A Data encryption method in Security cryptography algorithm". In: *IEEE* (2023). DOI: <https://doi-org.e.bibl.liu.se/10.1109/ICICAT57735.2023.10263684>.
- [4] Joakim Argillander, Alvaro Alarcón, Chunxiong Bao, Chaoyang Kuang, Gustavo Lima, Feng Gao, and Guilherme B. Xavier. "Quantum random number generation based on a perovskite light emitting diode". In: *Communications Physics* (2023). URL: <https://www.nature.com/articles/s42005-023-01280-3>.
- [5] Mohamed Yaqub A, Sudikshan S, Navin Balaji E, Adarsh A, and M. Gayathri. "Quantum Randomness and Key Hashing in SSH Authentication Across Asymmetric Cryptographic Algorithms". In: *IEEE* (2024). DOI: <https://doi-org.e.bibl.liu.se/10.1109/GCCIT63234.2024.10862579>.
- [6] Shuangyi Zhu, Yuan Ma, Xusheng Li; Jing Yang, and Jingqiang Lin; Jiwu Jing. "On the Analysis and Improvement of Min-Entropy Estimation on Time-Varying Data". In: *IEEE* 15 (2019). DOI: <https://doi.org/10.1109/TIFS.2019.2947871>.
- [7] Meltem Sönmez Turan, Elaine Barker, John Kelsey, Kerry A. McKay, Mary L. Baish, and Mike Boyle. "Recommendation for the Entropy Sources Used for Random Bit Generation". In: (2023). URL: <https://doi.org/10.6028/NIST.SP.800-90B>.
- [8] Thomas M. Cover and Joy A. Thomas. *Elements of Information Theory*. 2nd ed. Hoboken, NJ: Wiley-Interscience, 2006.

-
- [9] Shuangyi Zhu, Yuan Ma, Tianyu Chen, Jingqiang Lin, and JiwuJing. "Analysis and Improvement of Entropy Estimators in NIST SP 800-90B for Non-IID EntropySources". In: *(ISSN 2078-2489)* (2017). DOI: <https://doi.org/10.13154/tosc.v2017.i3.151-168>.
 - [10] Meltem Sönmez Turan, Elaine Barker, John Kelsey, and Kerry A. McKayand. "Recommendation for the Entropy Sources Used for Random Bit Generation". In: *NIST Special Publication 800-90B* 89.1 (2018). DOI: <https://doi.org/10.6028/NIST.SP.800-90B>.
 - [11] T. Chai and R. R. Draxler. "Root mean square error (RMSE) or mean absolute error (MAE)?Arguments against avoiding RMSE in the literature". In: *Copernicus Publications on behalf of the European Geosciences Union* (2014). DOI: [doi:10.5194/gmd-7-1247-2014](https://doi.org/10.5194/gmd-7-1247-2014).
 - [12] [cppreference.com. std::chrono::high_resolution_clock - cppreference.com](https://en.cppreference.com/w/cpp/chrono/high_resolution_clock). https://en.cppreference.com/w/cpp/chrono/high_resolution_clock. Accessed: 2025-06-03. 2025.



List of Figures

3.1	Runtime vs. Data Lengths	10
3.2	MAE values vs. Data Lengths	11
4.1	Estimated Min-Entropy vs. True Min-Entropy	13
4.2	Estimated Min-Entropy vs. Data Length	13
4.3	MAE comparison MultiMCW Estimator vs. Compression Estimator	14
4.4	Runtime comparison MultiMCW estimator vs. Compression estimator	15



List of Tables

3.1	Estimated and True Min-Entropy vs. Data Lengths	9
4.1	MAE comparison between MultiMCW and Compression estimators.	15
4.2	Time complexity comparison MultiMCW estimator vs. Compression estimator . .	16



A Algorithms

Algorithm 1 Recommendation for the Entropy Sources Used for Random Bit Generation [7]

Given the input $S = (s_1, \dots, s_L)$, where $s_i \in A = \{x_1, \dots, x_k\}$.

1. Let window sizes be $w_1 = 63$, $w_2 = 255$, $w_3 = 1023$, $w_4 = 4095$, and $N = L - w_1$. Let `correct` be an array of N Boolean values, each initialized to 0.
2. Let `scoreboard` be a list of four counters, each initialized to 0. Let `frequent` be a list of four values, each initialized to Null. Let `winner` = 1.
3. For $i = w_1 + 1$ to L :
 - a. For $j = 1$ to 4:
 - i. If $i > w_j$, let `frequentj` be the most frequent value in $(s_{i-w_j}, s_{i-w_j+1}, \dots, s_{i-1})$. If there is a tie, then the most frequent value that has appeared most recently is assigned to `frequentj`.
 - ii. Else, let `frequentj` = Null.
 - b. Let `prediction` = `frequentwinner`.
 - c. If (`prediction` = s_i), let `correcti` = 1.
 - d. Update the scoreboard. For $j = 1$ to 4:
 - i. If (`frequentj` = s_i), let `scoreboardj` = `scoreboardj` + 1.
 - ii. If `scoreboardj` $\geq$ `scoreboardwinner`, let `winner` = j .
4. Let C be the number of ones in `correct`.
5. Calculate the predictor's global performance as

$$P_{\text{global}} = \frac{C}{N}.$$

The upper bound of the 99% confidence interval on P_{global} , denoted P'_{global} , is calculated as

$$P'_{\text{global}} = \begin{cases} 1 - 0.01 \frac{1}{N}, & \text{if } P_{\text{global}} = 0, \\ \min\left(1, P_{\text{global}} + 2.576 \sqrt{\frac{P_{\text{global}}(1 - P_{\text{global}})}{N - 1}}\right), & \text{otherwise.} \end{cases}$$

where 2.576 corresponds to the $Z_{1-0.005}$ value.

6. Calculate the predictor's local performance, based on the longest run of correct predictions. Let r be one greater than the length of the longest run of ones in `correct`. Use a binary search to solve the following for P_{local} :

$$0.99 = \frac{1 - P_{\text{local}}^x}{(r + 1 - r x) q} \times \frac{1}{x^{r+1}},$$

where $q = 1 - P_{\text{local}}$ and $x = x_{10}$, derived by iterating the recurrence relation

$$x_j = 1 + q P_{\text{local}}^r x_{j-1}^{r+1},$$

For j from 1 to 10, and $x_0 = 1$. *Notethatsolvingfor* P_{local} using the logarithm of these equations is robust against overflows. Table 3 given in Appendix G.2 provides some pre-calculated values of P_{local} .

7. The min-entropy is the negative logarithm of the greater performance metric:

$$\text{min-entropy} = -\log_2(\max(P'_{\text{global}}, P_{\text{local}}^{1/k})).$$

Algorithm 2 Pseudocode for Compression estimator [7]

Given the input $S = (s_1, \dots, s_L)$, where $s_i \in A = \{0, 1\}$

1. Let $b = 6$. Create a new sequence, $S' = (s'_1, \dots, s'_{\lfloor L/b \rfloor})$, by dividing S into non-overlapping b -bit blocks. If L is not a multiple of b , discard the extra data.
2. Partition the dataset, S' , into two disjoint groups. These two groups will form the dictionary and the test data.
 - a. Create the dictionary from the first $d' = 1000$ elements of S' , $(s'_1, \dots, s'_{d'})$.
 - b. Use the remaining $v = \lfloor L/b \rfloor - d'$ observations, $(s'_{d'+1}, \dots, s'_{\lfloor L/b \rfloor})$, for testing.
3. Initialize the dictionary `dict` to an all zero array of size 2^b . For i from 1 to d' , let $d_{\text{icr}}[s'_i] = i$. The value of `dict` $[s'_i]$ is the index of the last occurrence of each s'_i in the dictionary.
4. Run the test data against the dictionary created in Step 2.
 - a. Let D be a list of length v .
 - b. For i from $d' + 1$ to $\lfloor L/b \rfloor$:
 - i. If `dict` $[s'_i] \neq 0$, then

$$D_{i-d'} = i - \text{dict}[s'_i].$$

Update the dictionary with the index of the most recent observation: `dict` $[s'_i] = i$.

- ii. If `dict` $[s'_i] = 0$, add that value to the dictionary, i.e. `dict` $[s'_i] = i$. Let $D_{i-d'} = i$.
5. Calculate the sample mean, $\bar{X}$, and sample standard deviation $\hat{\sigma}$, of $(\log_2(D_1), \dots, \log_2(D_v))$:

$$\bar{X} = \frac{\sum_{i=1}^v \log_2 D_i}{v}, \quad c = 0.5907, \quad \hat{\sigma} = c \sqrt{\frac{\sum_{i=1}^v (\log_2 D_i)^2}{v-1} - \bar{X}^2}.$$

6. Compute the lower-bound of the confidence interval for the mean:

$$\bar{X}' = \bar{X} - \frac{2.576 \hat{\sigma}}{\sqrt{v}}.$$

7. Using a binary search, solve for the parameter p in

$$\bar{X}' = G(p) + (2^b - 1)G(q),$$

where

$$G(z) = \frac{1}{v} \sum_{t=d'+1}^{\lfloor L/b \rfloor} \sum_{u=1}^t \log_2(u) F(z, t, u),$$

$$F(z, t, u) = \begin{cases} z^2(1-z)^{u-1}, & \text{if } u < t, \\ z(1-z)^{t-1}, & \text{if } u = t. \end{cases}$$

and

$$q = \frac{1-p}{2^b-1},$$

8. If the binary search yields a solution, then

$$\text{min-entropy} = -\log_2(p)/b.$$

If the search does not yield a solution, then

$$\text{min-entropy} = 1.$$

Algorithm 3 Pseudocode for synthetic data generation with known min-Entropy

```
1: Let  $L_{\text{bits}} \leftarrow n$ ,  $H_{\text{target}} \leftarrow t$ 
2: Compute  $p_{\text{max}} \leftarrow 2^{-H_{\text{target}}}$ 
3: Compute  $p_{\text{rest}} \leftarrow 1 - p_{\text{max}}$ 
4: Initialize random engine  $\text{rng}$  and Bernoulli distribution  $\text{dist}(p_{\text{max}})$ 
5: Initialize array  $\text{seq}[0 \dots L_{\text{bits}} - 1]$ 
6: for  $i = 0$  to  $L_{\text{bits}} - 1$  do
7:   if  $\text{dist}(\text{rng})$  is true then
8:      $\text{seq}[i] \leftarrow 0$ 
9:   else
10:     $\text{seq}[i] \leftarrow 1$ 
11:   end if
12: end for
13: Open file "../data_sets/syn_data_20.txt" for writing
14: for each  $\text{bit}$  in  $\text{seq}$  do
15:   Write  $\text{bit}$  to file (one per line)
16: end for
17: Close file
```
